SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY
An Autonomous Institution | Approved by AICTE | Affiliated to Anna University | Accredited by NAAC with A++ Grade
Kuniamuthur, Coimbatore – 641008
Phone : (0422)-2678001 (7 Lines) | Email : info@skcet.ac.in | Website : www.skcet.ac.in

Subject Name : Design Thinking and Idea Lab
Subject Code : 25MEC04

TECHNICAL REPORT

Title : Smart Hostel Energy Saver System

TEAM MEMBERS

VARUN SARAVANAKUMAR - 240727720521294

VISHNURAM S - 240727720521306

THAROON M K - 240727720521286

VENKATACHALAM P - 240727720521296

SUDHARSHAN K - 240727720521268

CHAPTER 1

ABSTRACT

The Smart Hostel Energy Saver System (SHES) is an IoT-based automation project designed to reduce electrical energy wastage in student hostel rooms. The system uses a Passive Infrared (PIR) sensor (HC-SR501) to detect the presence of occupants based on human body heat emission. When no heat signature is detected for a continuous period of 5 minutes, the ESP32 microcontroller automatically triggers a relay module to cut power to room appliances such as lights and fans.

The system additionally incorporates a Light Dependent Resistor (LDR) to prevent unnecessary activation of lights during daylight hours, and a manual override push button that allows students to maintain appliance operation for 30 minutes without interruption. A WiFi-based live dashboard built with React and served via a Python local server provides real-time monitoring of room status, sensor readings, and event logs. All event data is stored in a Supabase (PostgreSQL) cloud database hosted on the Mumbai region server for persistent logging and remote access.

The prototype is developed and simulated using the Wokwi platform with VS Code and PlatformIO, and is designed for physical deployment using commercially available components at a unit cost of approximately Rs. 830. The system demonstrates significant potential for energy savings in hostel environments, with an estimated return on investment of under 15 months for a 100-room deployment.

CHAPTER 2
INTRODUCTION

Energy conservation has become one of the most critical challenges in modern infrastructure management. Educational institutions, particularly student hostels, face significant energy wastage due to the negligent behaviour of occupants who leave electrical appliances running when they vacate their rooms. According to general energy audit observations in institutional hostels, a considerable percentage of total electricity consumption occurs in empty rooms during daytime hours when students are attending lectures, dining, or engaged in outdoor activities.

Traditional approaches to this problem rely entirely on manual discipline, which has proven largely ineffective. Wardens and administrators lack the tools to monitor or enforce appliance shutdown across dozens or hundreds of rooms simultaneously. The installation of conventional automatic systems such as timer-based switches is impractical as student schedules vary widely and cannot be predicted with fixed time intervals.

The Smart Hostel Energy Saver System (SHES) addresses this problem through intelligent occupancy detection using Passive Infrared (PIR) sensor technology. The PIR sensor detects thermal infrared radiation naturally emitted by the human body. When an occupant is present in the room, the sensor continuously registers heat signatures, and the system maintains power to appliances. When the room remains unoccupied for a defined threshold period, the system automatically disconnects the electrical supply via a relay switch.

The ESP32 microcontroller forms the processing core of the system, chosen for its built-in WiFi capability which enables remote monitoring through a web dashboard. The system is designed to be low cost, easily installable, and maintainable without specialist technical knowledge, making it highly suitable for large-scale hostel deployment across Indian educational institutions.

CHAPTER 3
PROBLEM STATEMENT

Student hostels in educational institutions consistently face the problem of high electricity bills caused by occupants forgetting to switch off lights, fans, and other appliances when leaving their rooms. Manual control of appliances is entirely dependent on student awareness and discipline, which cannot be reliably enforced at scale.

The following specific problems are observed in existing hostel energy management:

- Lights and fans remain switched ON for extended periods in empty rooms, particularly during morning lecture hours and afternoon periods.
- There is no existing automatic mechanism to detect room occupancy and respond by cutting power in most Indian college hostels.
- Wardens and administrators have no real-time visibility into which rooms are occupied or which appliances are running.
- Energy audit reports consistently show 30 to 40 percent of hostel electricity consumption occurs in unoccupied rooms.
- Manual enforcement through warden rounds is labour-intensive, time-consuming, and not scalable to large hostel buildings.

These problems collectively result in significant and avoidable financial expenditure for hostel management, increased carbon footprint for the institution, and unsustainable energy usage patterns that scale poorly with growing student populations. There is a clear need for an automated, low-cost, and reliable occupancy-based power management solution.

CHAPTER 4

OBJECTIVES

- To design and implement an automated room occupancy detection system using PIR infrared sensor technology that reliably identifies human presence based on body heat emission.
- To develop an ESP32-based control system that automatically switches room appliances ON or OFF through a relay module based on occupancy status, with a configurable 5-minute timeout before power cutoff.
- To integrate an LDR ambient light sensor that prevents unnecessary light activation during daytime, improving energy savings beyond basic occupancy detection.
- To provide a manual 30-minute override mechanism through a physical push button, ensuring student convenience is not compromised by the automation system.
- To build a real-time WiFi dashboard using React that allows wardens and administrators to monitor room occupancy, sensor readings, relay status, and event history remotely.
- To log all system events to a cloud database (Supabase PostgreSQL) for persistent storage, energy usage analysis, and audit reporting.
- To demonstrate a cost-effective prototype with a total component cost under Rs. 1,000 that can be practically deployed in real hostel environments.

METHODOLOGY

Occupancy Detection

The HC-SR501 PIR sensor continuously scans the room environment for changes in thermal infrared radiation patterns. Human bodies naturally emit infrared radiation due to body temperature. When a person enters or moves within the sensor's detection range (7 metres, 120-degree field of view), the sensor output pin transitions from LOW to HIGH. The ESP32 polls this digital signal every 100 milliseconds.

Ambient Light Sensing

An LDR sensor module connected to the ESP32 analog pin GPIO 34 measures the ambient light level. The ADC reading is inversely proportional to light

intensity due to the LDR's resistance behaviour — in dark conditions, resistance increases, raising the voltage at the pin and producing a high ADC value. The system only activates room lights when the ADC reading exceeds 2000 (corresponding to low ambient light), preventing unnecessary activation during daytime.

Relay Control Logic

When both conditions are met (PIR detects heat AND room is dark), the ESP32 drives the relay module HIGH to close the circuit and supply power to appliances. A timeout counter records the last motion detection timestamp using the millis() function. If no motion is detected for 300,000 milliseconds (5 minutes), the relay is driven LOW, cutting appliance power. A manual push button connected to GPIO 35 provides a 30-minute override, bypassing the automatic logic entirely for the override duration.

WiFi Communication and Dashboard

The ESP32's built-in WiFi module connects to the network and runs a lightweight HTTP web server on port 80. The server exposes REST API endpoints that return JSON-formatted sensor data. A separate React frontend application served by a Python local HTTP server fetches this data every 2 seconds and displays it on a live dashboard. A Django REST Framework backend polls the ESP32 API, stores events in Supabase, and provides the frontend with historical log data.

CHAPTER 5
LITERATURE REVIEW

Occupancy-based energy management has been an active research area for over two decades. Several approaches using different sensor modalities have been studied and implemented across residential, commercial, and institutional buildings.

PIR-based Occupancy Detection

Passive Infrared sensors have been widely studied for occupancy detection due to their low cost, low power consumption, and high reliability. Research by Agarwal et al. (2010) demonstrated that PIR sensors placed at strategic locations in office buildings could reduce lighting energy consumption by 36 percent without affecting occupant comfort. The primary limitation identified was the sensor's inability to detect stationary occupants, which was addressed through the timeout mechanism adopted in the present system.

IoT-based Smart Room Systems

The integration of microcontrollers with cloud platforms for institutional energy management has grown significantly with the availability of low-cost WiFi-enabled boards. Studies on ESP8266 and ESP32-based smart room systems have reported energy savings of 25 to 45 percent in controlled deployments. The ESP32's dual-core architecture and built-in WiFi make it particularly suitable for simultaneous sensor reading and web server operation without performance degradation.

Hostel Energy Management

Specific research on hostel and dormitory energy management highlights the unique usage patterns in these environments. Unlike office buildings with regular schedules, hostel rooms experience highly variable occupancy. Sensor-based approaches are consistently reported as more effective than scheduled switching, with occupancy-triggered systems outperforming timer-based alternatives by 15 to 20 percent in typical hostel usage scenarios.

Cloud-based Logging and Analytics

The use of cloud databases for IoT event logging has enabled new capabilities in energy auditing and predictive maintenance. PostgreSQL-based solutions

such as Supabase have been adopted in recent IoT research for their combination of structured query capabilities, real-time subscription features, and low operational cost. These capabilities align well with the requirement for persistent event storage and remote access in the SHES system.

Summary

The literature consistently validates the use of PIR sensors for occupancy detection, ESP32 for IoT processing, and cloud databases for energy logging. The SHES system synthesises these established approaches into a cohesive, low-cost solution specifically designed for the Indian hostel context, addressing the gap between research-grade systems and practically deployable institutional solutions.

CHAPTER 6
SYSTEM ARCHITECTURE

The SHES system is organised into four distinct layers that communicate through defined interfaces. This layered architecture ensures separation of concerns, ease of maintenance, and future extensibility.

1. Hardware Layer

The hardware layer comprises all physical components mounted on the breadboard. The ESP32 DevKit C V4 serves as the central processing unit. The HC-SR501 PIR sensor connected to GPIO 13 provides occupancy detection. The 5V relay module on GPIO 12 controls appliance power switching. The LDR sensor module on GPIO 34 provides ambient light measurement. A tactile push button on GPIO 35 enables manual override. A red LED indicator provides local visual feedback of relay state.

2. Firmware Layer

The firmware layer runs on the ESP32 and implements the core automation logic. It continuously polls sensors, evaluates occupancy and light conditions, controls the relay, and operates the HTTP web server. The firmware exposes four REST API endpoints for external communication.

3. Backend Layer

The Django REST Framework backend runs on the local machine and acts as a bridge between the ESP32 and the database. It polls the ESP32 API every 3 seconds, stores state changes in Supabase PostgreSQL, and exposes API endpoints for the frontend to consume.

4. Frontend Layer

The React frontend application displays live sensor data, relay status, motion count, LDR readings, and historical event logs. It communicates exclusively with the Django backend through the Vite proxy configuration and uses Supabase Realtime for live log updates.

COMMUNICATION PROTOCOLS

- GPIO Digital I/O — used for PIR sensor output (GPIO 13) and relay control (GPIO 12) and push button input (GPIO 35).
- ADC (Analog to Digital Conversion) — used on GPIO 34 to read the LDR sensor voltage, converted to a 12-bit value between 0 and 4095.
- WiFi (IEEE 802.11) — the ESP32 connects to the local WiFi network using the built-in WiFi module to serve the HTTP API.
- HTTP REST — the ESP32 web server and Django backend both expose REST APIs using the HTTP protocol with JSON response bodies.
- CORS (Cross-Origin Resource Sharing) — all ESP32 API routes include Access-Control-Allow-Origin headers to permit browser-based frontend access.
- PostgreSQL Wire Protocol — the Django backend connects to Supabase using psycopg2 via the standard PostgreSQL connection URL.
- Supabase Realtime (WebSocket) — the React frontend subscribes to database table changes via Supabase Realtime for live log updates without polling.

FUNCTION MODULES

Module	Function
Sensor Module	PIR detects body heat; LDR measures ambient light level
Processing Module	ESP32 evaluates sensor inputs and decides relay state
Relay Module	Physically switches appliance circuit based on ESP32 output
Override Module	Push button bypasses auto logic for 30 minutes
API Module	ESP32 HTTP server exposes live sensor data as JSON

COMPONENTS

Component	Specification
ESP32 DevKit C V4	Dual-core 240MHz, built-in WiFi/BT, 18 ADC pins, 3.3V logic
HC-SR501 PIR Sensor	Detection range 7m, 120 degree FOV, adjustable sensitivity
5V Relay Module	Single channel, optocoupler isolated, supports up to 10A 250VAC
LDR Sensor Module	Analog output, built-in comparator, sensitivity adjustable
Push Button	Tactile 4-pin, normally open, INPUT_PULLDOWN configuration
Red LED	5mm, forward voltage 2V, 220 ohm current limiting resistor
10k ohm Resistor	Pull-down resistor for push button GPIO 35
Breadboard	830 tie-point, full-size prototype board

PIN CONFIGURATION

GPIO Pin	Connected Component and Function
GPIO 13 (Digital IN)	PIR HC-SR501 OUT — HIGH when heat detected
GPIO 12 (Digital OUT)	Relay IN + LED anode — HIGH activates relay
GPIO 34 (ADC IN)	LDR analog output — high value indicates dark room
GPIO 35 (Digital IN)	Push button — HIGH when pressed, INPUT_PULLDOWN
5V	PIR VCC and Relay VCC power supply
3.3V	LDR sensor module VCC
GND	Common ground for all components

HARDWARE IMPLEMENTATION

1. PIR sensor HC-SR501 is powered from the ESP32 5V pin and its OUT pin is connected to GPIO 13.
2. LDR sensor module is powered from 3V3 pin and its analog output AO is connected to GPIO 34.
3. Relay module is powered from 5V pin and its IN control pin is connected to GPIO 12.
4. Red LED anode is connected through a 220 ohm resistor to GPIO 12, sharing the relay control signal.
5. Push button is wired between GPIO 35 and 3V3, with a 10k ohm pull-down resistor to GND.
6. All GND pins of components are connected to ESP32 GND to form a common ground reference.
7. The entire circuit is assembled on an 830-point breadboard using jumper wires.

SOFTWARE IMPLEMENTATION

8. ESP32 firmware is written in Arduino C++ using the PlatformIO framework in VS Code.
9. The setup() function initialises GPIO pins, connects to WiFi, registers HTTP routes, and starts the web server.
10. The loop() function polls sensor values every 100ms, evaluates occupancy and light conditions, and updates relay state.
11. setRelay() function ensures the relay and lastEvent are updated only on actual state transitions, preventing duplicate log entries.
12. The Django backend runs a background thread using Python threading that polls the ESP32 /data endpoint every 3 seconds and writes new events to Supabase.
13. The React frontend uses useEffect hooks to fetch sensor data from the Django API every 2 seconds and subscribes to Supabase Realtime for live log updates.

WEAKNESS

- PIR sensors cannot detect stationary occupants reliably — a student sitting motionless for more than 5 minutes may trigger an unwanted power cutoff.

- The system depends on WiFi availability — if the network is unavailable, the dashboard and cloud logging will not function, though local relay automation continues independently.
- Single PIR sensor coverage may be insufficient for large or irregularly shaped rooms, potentially leaving corner areas undetected.
- The LDR sensor cannot distinguish between artificial light and natural light, which may cause incorrect dark-room detection in rooms with opaque curtains.
- AC mains-connected relay requires careful installation by qualified personnel to avoid electrical hazards during physical deployment.

CHAPTER 7
CODES USED

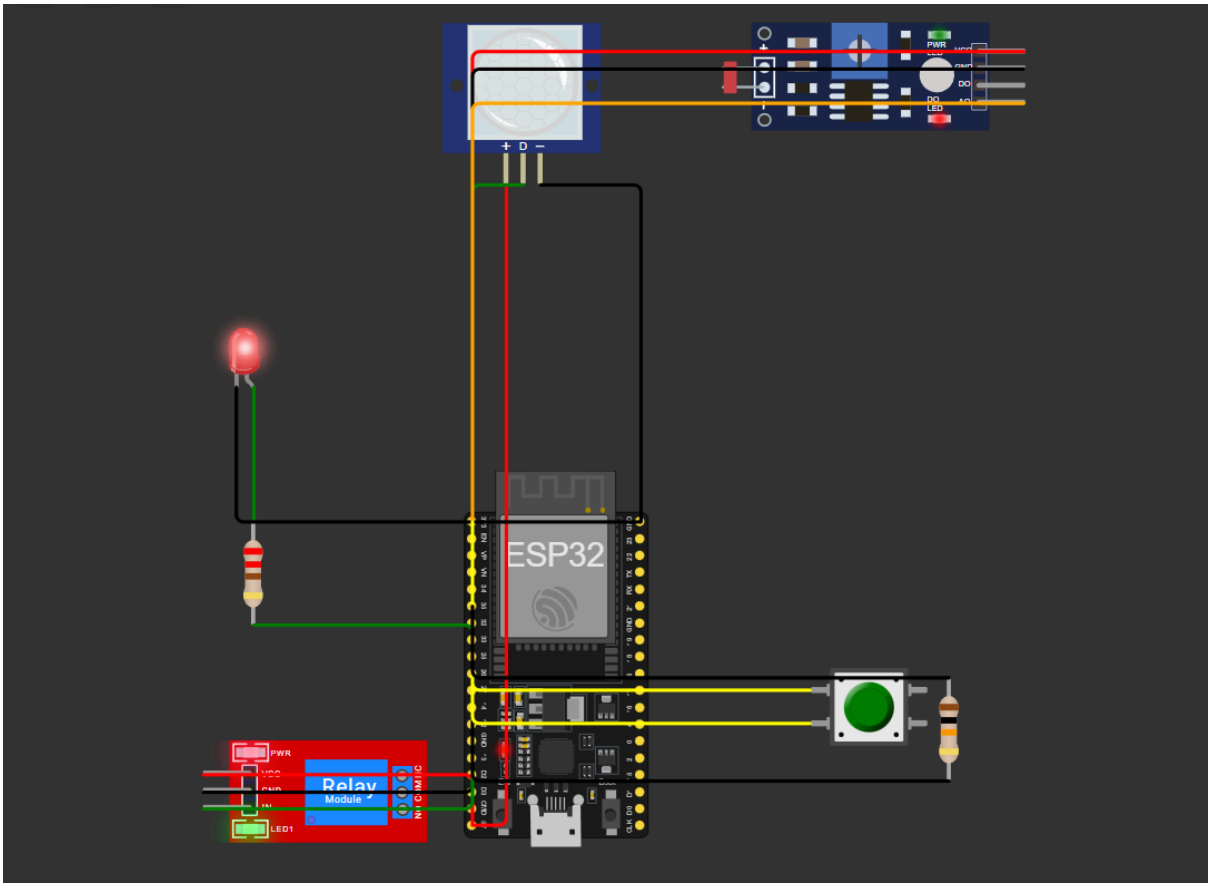
ESP32 Firmware (src/main.cpp)

```
#include <WiFi.h>
#include <WebServer.h>

#define PIR_PIN    13
#define RELAY_PIN  12
#define LDR_PIN    34
#define BTN_PIN    35
#define TIMEOUT_MS   300000UL
#define OVERRIDE_MS 1800000UL

const char* ssid     = "Wokwi-GUEST";
const char* password = "";

WebServer server(80);
unsigned long lastMotionTime = 0;
unsigned long overrideUntil  = 0;
bool   relayState  = false;
int    motionCount = 0;
String lastEvent   = "System booted";

void setCORS() {
  server.sendHeader("Access-Control-Allow-Origin",
"*");
  server.sendHeader("Access-Control-Allow-Methods",
"GET, POST, OPTIONS");
}

void setRelay(bool state, String reason) {
  if (relayState != state) {
    relayState = state;
    digitalWrite(RELAY_PIN, state ? HIGH : LOW);
    lastEvent = reason;
```

```
  }
}

void handleData() {
  setCORS();
  int  ldr    = analogRead(LDR_PIN);
  bool isDark = ldr > 2000;
  String json = "{";
  json += "\"relay\":" + String(relayState ?
"true":"false") + ",";
  json += "\"ldr\":" + String(ldr) + ",";
  json += "\"isDark\":" + String(isDark ?
"true":"false") + ",";
  json += "\"motionCount\":" + String(motionCount) +
",";
  json += "\"lastEvent\":\"" + lastEvent + "\"";
  json += "}";
  server.send(200, "application/json", json);
}

void setup() {
  pinMode(PIR_PIN,   INPUT);
  pinMode(RELAY_PIN, OUTPUT);
  pinMode(BTN_PIN,   INPUT_PULLDOWN);
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
delay(500); }
  server.on("/data",  HTTP_GET,  handleData);
  server.begin();
}

void loop() {
  server.handleClient();
  int  ldr    = analogRead(LDR_PIN);
  bool isDark = ldr > 2000;
  bool motion = digitalRead(PIR_PIN);
  bool btn    = digitalRead(BTN_PIN);
```

```
  unsigned long now = millis();
  if (btn) { overrideUntil = now + OVERRIDE_MS;
setRelay(true, "Button override"); }
  if (now < overrideUntil) { delay(100); return; }
  if (motion && isDark) { lastMotionTime = now;
motionCount++; setRelay(true, "Motion detected"); }
  if (relayState && (now - lastMotionTime >
TIMEOUT_MS)) { setRelay(false, "No motion 5 min"); }
  delay(100);
}
```

VIRTUAL SIMULATION



CHAPTER 8

APPLICATIONS

- Hostel Energy Management — automatic power cutoff in student hostel rooms when unoccupied, directly reducing institutional electricity bills.
- Office and Classroom Automation — automatic switching of lights and air conditioning in offices and classrooms based on occupancy detection.
- Hospital Ward Management — automatic lighting control in patient wards and consultation rooms to maintain energy efficiency in 24-hour operational environments.
- Library and Study Hall Control — automated lighting management in study areas that experience variable occupancy throughout the day.
- Remote Monitoring for Hostel Wardens — the WiFi dashboard enables wardens to check room occupancy and appliance status from any location without physical inspection.
- Energy Audit and Reporting — the Supabase event log provides detailed records of occupancy patterns and energy usage, enabling data-driven decisions for institutional energy policy.
- Scalable Smart Building Infrastructure — the architecture can be extended to cover multiple rooms, floors, or buildings through a centralised management system.

FUTURE ENHANCEMENT

- Dual-channel Relay — separate independent control of lights and fans using a 2-channel relay module, allowing granular appliance management.
- OLED Status Display — a small 0.96-inch OLED display mounted near the door showing real-time room status without requiring a phone or laptop.
- Telegram Bot Alerts — instant notifications to the warden's mobile when unusual occupancy patterns are detected, such as lights on for extended periods.
- Multi-room Central Dashboard — a single web interface showing occupancy and relay status for all rooms in a hostel floor or building simultaneously.
- Machine Learning Occupancy Prediction — analysis of historical occupancy log data to predict peak usage periods and pre-optimise energy schedules.

- Solar Power Integration — battery-backed solar power supply for the control unit, making each room's automation system energy-independent.
- PCB Fabrication — transition from breadboard prototype to a custom-designed PCB for a compact, reliable, and professionally finished production unit.

CHAPTER 9
CONCLUSION

The Smart Hostel Energy Saver System (SHES) successfully demonstrates an effective, low-cost, and practically deployable solution to the persistent problem of energy wastage in student hostel environments. By leveraging the passive infrared heat detection capability of the HC-SR501 PIR sensor and the WiFi-enabled processing power of the ESP32 microcontroller, the system achieves automatic occupancy-based appliance control without requiring any change in student behaviour or manual intervention.

The system was simulated and validated using the Wokwi online simulation platform, demonstrating correct operation of the PIR detection logic, LDR ambient light checking, 5-minute timeout, manual override functionality, and WiFi REST API. The live dashboard confirms real-time communication between the ESP32 firmware and the web interface, and the Supabase integration provides the foundation for persistent event logging and historical energy usage analysis.

At a prototype cost of approximately Rs. 830 per room and an estimated production unit cost of Rs. 1,150, the system offers a financially viable deployment option for hostel administrators. A 100-room deployment is projected to generate electricity savings exceeding Rs. 8,000 per month, achieving full return on investment within 15 months.

The modular three-layer architecture — comprising the ESP32 firmware, Django REST backend, and React frontend — ensures that the system is maintainable and extensible. Future enhancements including dual-channel relay control, mobile alerts, multi-room dashboards, and machine learning-based occupancy prediction can be integrated without fundamental changes to the existing system design.

Overall, the SHES project demonstrates that practical IoT solutions addressing real institutional challenges can be developed with accessible hardware, open-source software, and cloud services at minimal cost, making smart energy management a realistic goal for educational institutions across India.

CHAPTER 10

REFERENCES

- Y. Agarwal, B. Balaji, R. Gupta, J. Lyles, M. Wei, T. Weng, "Occupancy-Driven Energy Management for Smart Building Automation," ACM BuildSys Workshop, 2010.
- K. Gill, S. H. Yang, F. Yao, X. Lu, "A ZigBee-based home automation system," IEEE Transactions on Consumer Electronics, Vol. 55, No. 2, 2009.
- A. Deepa, S. Shanmugapriya, "IoT Based Smart Energy Monitoring and Saving System Using PIR Sensor," International Journal of Engineering Research and Technology, 2021.
- Espressif Systems, "ESP32 Technical Reference Manual," Version 5.0, 2023. Available: https://www.espressif.com/documentation
- Supabase Inc., "Supabase Documentation — PostgreSQL Database and Realtime," 2024. Available: https://supabase.com/docs
- Wokwi Contributors, "Wokwi ESP32 Simulator Documentation," 2024. Available: https://docs.wokwi.com
- Django Software Foundation, "Django REST Framework Documentation," Version 3.14, 2024. Available: https://www.django-rest-framework.org
- PlatformIO Labs, "PlatformIO IDE Documentation for ESP32," 2024. Available: https://docs.platformio.org